Student Name: Kushagra Vashistha **UID: 24BAI70915**
Branch: AIT-CSE (Cyber Security) **Section/Group: 24AIT_KRG-G2**
Semester: 4 **Date of Performance: 01/04/2026**
Subject Name: Database Management System **Subject Code: 24CSH-298**

WORKSHEET-8

AIM: To design and implement a stored procedure that accepts gender as an input parameter and returns the total number of employees based on the gender passed, thereby demonstrating procedural programming and parameterized logic in PL/SQL.

S/W Requirement:
- Database Management System: PostgreSQL / Oracle Database Express Edition
- Database Administration Tool: pgAdmin

OBJECTIVES:

-Understand the structure and components of a stored procedure.

-Implement **IN** and **OUT** parameters in PL/SQL procedures.

-Pass dynamic input values to stored procedures.

-Apply aggregate functions within procedures for data analysis.

PROBLEM STATEMENT:

Organizations often require flexible reporting mechanisms to analyze workforce distribution based on gender for compliance and internal analytics. A reusable database logic is needed to dynamically compute these counts based on user input.

TABLE STRUCTURE:

```sql
CREATE TABLE employeedata(
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    gender VARCHAR(10),
    salary NUMERIC(10,2)
);
```

```sql
INSERT INTO employeedata(emp_name, gender, salary)
VALUES
('Amit', 'Male',30000),
('Riya', 'Female',35000),
('John', 'Male',28000),
('Sneha', 'Female',40000);
```

1. PROBLEM STATEMENT – Stored Procedure Design

Create a stored procedure that accepts a gender string as input and returns the count of employees matching that gender.

PROGRAM:

```sql
CREATE OR REPLACE PROCEDURE count_emp(
    IN p_gen VARCHAR(20),
    OUT emp_count INT,
    INOUT status VARCHAR
)
AS $$
BEGIN
    SELECT COUNT(*)
    INTO emp_count
    FROM employeedata
    WHERE gender = p_gen;

    status := 'SUCCESS';
END;
$$ LANGUAGE plpgsql;
```

OUTPUT:

```
CREATE PROCEDURE

Query returned successfully in 168 msec.
```

2. PROBLEM STATEMENT – Procedure Execution

Execute the stored procedure using a PL/SQL block to retrieve and display the count for a specific gender (e.g., 'Male').

PROGRAM:

```
DO $$
DECLARE
    GEN VARCHAR(20) := 'Male';
    COUNT_BY_GEN INT;
    STATUS VARCHAR(20) := 'Fail';
BEGIN

    CALL COUNT_EMP(GEN, COUNT_BY_GEN, STATUS);

    RAISE NOTICE
    'YOUR GENDER IS % YOUR COUNT IS % AND STATUS IS %',
    GEN, COUNT_BY_GEN, STATUS;

END;
$$;
```

OUTPUT:

```
NOTICE:  YOUR GENDER IS Male YOUR COUNT IS 2 AND STATUS IS SUCCESS
DO
```

<u>LEARNING OUTCOMES:</u>

1. Understood the structure and components of a stored procedure.
2. Learned to implement **IN** parameters for input and **OUT** parameters for returning values.
3. Successfully passed dynamic input values to filter database results.
4. Applied the COUNT() aggregate function within a procedural block.
5. Developed reusable database logic applicable to real-world scenarios at companies like CData and Toddle.

<u>CONCLUSION:</u>

This experiment demonstrated the effectiveness of parameterized stored procedures in PL/SQL. By using **IN** and **OUT** parameters, we created a flexible and reusable logic that can dynamically analyze workforce data, improving efficiency and reducing code redundancy in database management systems.